

00 - Genel Repo Denetimi

Olusturulma: 14 Nisan 2026 19:49 Kaynak: docs/audit Format: Tekil rapor

00 - Genel Repo Denetimi

Tarih: 2026-04-13 Kapsam: Genel repo taraması, mimari çıkarımı, kritik akışların ilk değerlendirmesi. Sınır: Bu raporda kod davranışı değiştirilmedi; yalnızca denetim dokümanı oluşturuldu.

1. Proje özeti

Bu repo, restoran POS kullanım senaryosu için masa, sipariş, mutfak, ödeme, müşteri, stok, rezervasyon, caller ID ve yazıcı akışlarını kapsayan masaüstü/web hibrit bir üründür.

Ana mimari:

- Frontend: React 18 + Vite. Giriş dosyaları `client/src/main.jsx` ve `client/src/App.jsx`. Route/top-level ekran orkestrasyonu `client/src/App.jsx:46` sonrası tek dosyada yoğunlaşmış.
- Backend: Node.js + Express + SQLite. API entrypoint `server/index.js`; route kayıtları `server/index.js:111-127`.
- Veritabanı: `better-sqlite3`, WAL modu ve foreign key açık. Kanıt: `server/config/database.js:11-16`.
- Desktop shell: Electron, Express backend'i child process olarak başlatıyor. Kanıt: `electron/main.cjs:1-5`, child env kurulumu `electron/main.cjs:170-204`.
- Yerel donanım köprüsü: `store-bridge`, API'den print job poll ediyor ve USB/network yazıcıya ESC/POS buffer gönderiyor. Kanıt: `store-bridge/jobs/poller.js:5-13`, `store-bridge/jobs/poller.js:72-99`.
- Yazdırma modeli: POS backend `print_jobs` üretir; StoreBridge claim edip fiziksel yazıcıya yollar. Kanıt: `server/services/printJobs.js:50-82`, `store-bridge/jobs/poller.js:19-31`.

Kritik iş akışları:

- Auth: JWT bearer token, kullanıcı/rol DB'den okunuyor. Kanıt: `server/middleware/auth.js:5-33`.
- Business scoping: route middleware `req.businessId` atıyor. Kanıt: `server/middleware/auth.js:45-49`.
- Sipariş: `server/routes/orders.js` hem listeleme, paket sipariş, mutfak job üretimi, item lifecycle gibi çok sayıda davranışı taşıyor. Kanıt: `server/routes/orders.js:1-18`, `server/routes/orders.js:52-67`.
- Ödeme: tekil ve parçalı ödeme, masa kapatma, müşteri istatistiği ve fiş job üretimi aynı route dosyasında. Kanıt: `server/routes/payments.js:164-183`, `server/routes/payments.js:197-240`.
- Yazıcı kod sayfası: StoreBridge renderer içinde PC857 ve Windows-1254 desteği var. Kanıt: `store-bridge/printers/renderers.js:16-28`, `store-bridge/printers/renderers.js:38-44`, `store-bridge/printers/renderers.js:115-154`, `store-bridge/printers/renderers.js:278-287`.

2. Güçlü yönler

- P1 olumlu: Yazdırma, doğrudan HTTP mock'tan job queue + bridge modeline taşınmış görünüyor. `client/src/services/api.js:195-199` eski `/api/print/*` uçlarının deprecated olduğunu açıkça söylüyor; aktif akışın `print_jobs + StoreBridge` olduğu not edilmiş. Bu doğru mimari yöndür.
- P1 olumlu: Print job idempotency düşünülmüş. `server/services/printJobs.js:63-80` `INSERT OR IGNORE` ve `idempotency_key` kullanıyor; mutfak job'larında hash tabanlı anahtar var (`server/services/printJobs.js:168-173`).
- P1 olumlu: Backend startup'ta production JWT secret zorunluluğu var. `server/config/index.js:22-27` production'da fallback secret ile açılışı engelliyor.
- P1 olumlu: API 404'leri JSON dönüyor; SPA fallback'in API hatalarını HTML'e çevirmemesi için özel blok var. Kanıt: `server/index.js:129-139`.
- P2 olumlu: Server test klasörü mevcut ve iş kritik alanlardan bazılarını kapsıyor: auth middleware, validation, migration idempotency, order transaction, printer encoding, print job idempotency ve integration testleri. Kanıt: `server/tests/*.test.js`.
- P2 olumlu: Client build üretilebiliyor. `npm run build --prefix client` başarılı bitti; ancak bundle boyutu uyarısı var.

3. Kritik problemler

P0 - Net P0 bulunmadı

Bu ilk genel taramada uygulamayı doğrudan veri kaybına, yetkisiz toplu erişime veya açılışın tamamen imkansız hale gelmesine götüren kanıtlanmış P0 saptanmadı. Ancak aşağıdaki P1'ler üretim kalitesi için sert biçimde ele alınmalı.

P1 - Server test komutu çalışmıyor

Kanıt:

- Root test script'i server testine delegasyon yapıyor: `package.json:13`.
- Server test script'i `vitest run`: `server/package.json:9`.
- `vitest` devDependency olarak tanımlı: `server/package.json:27-30`.
- Çalıştırılan komut sonucu: `npm test --prefix server -> 'vitest' is not recognized as an internal or external command`.

Etki: Repo testleri varmış gibi görünüyor ama mevcut çalışma ortamında test kapısı kırık. Bu, yapılan her düzeltmenin regresyon riskini yükseltir. Özellikle POS, ödeme ve yazdırma gibi para/donanım akışlarında kabul edilemez.

Öneri: `server/node_modules` bağımlılıklarını temiz ve eksiksiz kur; CI/yerel doğrulama için root `npm test` komutunu güvenilir hale getir. Ek olarak lockfile ve workspace stratejisini sadeleştir; şu anda root, client, server, store-bridge ayrı package-lock ile yönetiliyor.

P1 - Monolitik dosyalar ürün kalitesini düşürüyor

Kanıtlanan büyük dosyalar:

- `client/src/components/orders/OrderScreen.jsx`: 1428 satır.

- `store-bridge/printers/renderers.js` : 1271 satır.
- `client/src/components/settings/PrinterDetailPage.jsx` : 972 satır.
- `electron/main.cjs` : 860 satır.
- `server/migrations/run.js` : 673 satır.
- `server/routes/orders.js` : 627 satır.
- `client/src/components/payments/PaymentScreen.jsx` : 497 satır.
- `server/routes/payments.js` : 389 satır.

Etki: Bu dosyalar tek sorumluluk ilkesini zorluyor. POS ekranı, ödeme, yazıcı ayarı ve ESC/POS rendering gibi en riskli alanlar aynı zamanda en büyük ve en kırılgan dosyalar. Yeni özellik ekleme maliyeti artar; küçük düzeltmeler yan etki üretir.

Öneri: Öncelik sırası: `OrderScreen.jsx`, `renderers.js`, `PrinterDetailPage.jsx`, `electron/main.cjs`. Her biri için önce davranışı sabitleyen test/snapshot, sonra küçük servis/hook/component ayrıştırması yapılmalı.

P1 - Yazdırma karakter seti karmaşıklığı tek dosyada yoğun ve cihaz bağımlı

Kanıt:

- PC857 ESC t kod sayfası sabiti: `store-bridge/printers/renderers.js:16-22`.
- Windows-1254 ESC t sabiti: `store-bridge/printers/renderers.js:23-28`.
- `encodeWin1254` TL fallback ile çalışıyor: `store-bridge/printers/renderers.js:38-44`.
- PC857 manuel tablo içeriyor: `store-bridge/printers/renderers.js:67-106`.
- Runtime encoding mode çözümü `win1254` default: `store-bridge/printers/renderers.js:278-287`.
- Bridge, receipt veya win1254 için `escT = 32` ve `skipPhoenixCmd = true` zorluyor: `server/routes/bridge.js:47-53`.

Etki: Türkçe karakter bozulması tek bir yerde değil; backend printer option normalizasyonu, bridge renderer, fiziksel yazıcı firmware'i ve env/pos-config override zincirinin birleşiminde oluşabilir. Kodda farklı cihazlar için doğru düşünülmüş parçalar var, fakat tek büyük dosyada ve env tabanlı koşullarla yönetildiği için hata ayıklama maliyeti yüksek.

Öneri: Yazdırma için ayrı bir `encoding` modülü çıkar; PC857/WIN1254 encode testlerini hem byte snapshot hem ESC/POS komut snapshot olarak büyüt. Her fiziksel yazıcı profili için `encodingMode`, `escT`, `skipPhoenixCmd`, `lineWidth` değerleri tek profil objesinde tutulmalı.

P1 - Paketleme çıktıları ve node_modules repo içinde taramayı kısıyor

Kanıt:

- Repo kökünde `dist-electron`, `node_modules`, `client/node_modules`, `server/node_modules`, `store-bridge/node_modules` mevcut.
- Dosya tarama sırasında `server/node_modules/restoran-pos/...` altında recursive `file:..` bağlantıları nedeniyle PowerShell `Could not find a part of the path ... node_modules\restoran-pos\...` hataları verdi.
- `client/package.json:17`, `server/package.json:22`, `store-bridge/package.json` içinde `restoran-pos: file:..` bağımlılığı var.

Etki: Yerel paket bağımlılığı repo kökünü alt paketlerin `node_modules` içine tekrar taşıyor. Bu, taramayı, paketlemeyi, antivirüs/Windows path davranışını ve CI sürelerini gereksiz kırılgan hale getirir.

Öneri: npm workspaces veya açık paket sınırı kullan. Alt paketlerin root projeyi dependency olarak çekmesi gerçekten gerekiyorsa bunun nedenini belgeleyip publish/workspace protokolüne taşı.

P1 - Frontend bundle uyarısı: tek parça uygulama büyüyor

Kanıt:

- `npm run build --prefix client` başarılı.
- Vite uyarısı: minified chunk 500 kB üstü.
- Üretilen ana JS: `assets/index-CwuMQpUU.js` 948.07 kB, gzip 264.37 kB.
- XLSX chunk: `assets/xlsx-D_0l8YDs.js` 429.03 kB, gzip 143.08 kB.

Etki: POS genelde düşük donanımlı Windows kasalarda çalışır. Büyük ilk yükleme, Electron içinde de startup ve memory baskısı yaratır.

Öneri: Route bazlı lazy loading; müşteri import/export için `xlsx` modülünü yalnız ilgili ekranda dynamic import ile yükle; rapor/chart ekranlarını ayrı chunk'a ayır.

4. Amatör görünen alanlar

- P1: `OrderScreen.jsx` 1428 satır. Sipariş ekranı bir ekran bileşeni olmaktan çıkıp iş kuralı, navigasyon, state, API orchestration ve UI davranışını aynı yerde toplama riski taşıyor. Bu profesyonel POS ürününde bakım sorunudur.
- P1: `store-bridge/printers/renderers.js` 1271 satır. Encoding, layout, ESC/POS komutları, metin sarma, diagnostik ve template render aynı dosyada. Yazıcı sorunu çıktığında nokta atışı izolasyon zorlaşır.
- P1: `electron/main.cjs` 860 satır. Config okuma, JWT üretimi, DB taşıma, backend process, bridge process, caller ID helper ve pencere yönetimi aynı dosyada. Masaüstü lifecycle hataları bu yapıda pahalı olur.
- P2: `client/src/App.jsx:46-245` top-level route, auth role, modal state, quick payment state ve navigation fallback'lerini aynı komponentte yönetiyor. Şimdilik çalışabilir ama büyüme yönü yanlış.
- P2: API client tek sınıfta tüm domainleri topluyor. `client/src/services/api.js:77-210` auth, masa, ürün, sipariş, ödeme, müşteri, caller ID, rapor, stok, waiter call ve print uçlarını aynı dosyaya yığmış.
- P2: Root scriptlerde Windows `.bat`, cross-env, Electron paketleme ve server/client komutları karışık. `package.json:8-37` tek dosyada çok sayıda operasyonel sorumluluk taşıyor.

5. Teknik borç alanları

- P1: Test altyapısı yerelde kırık. Test dosyaları var ama `vitest` bulunmadığı için çalıştırılmıyor.
- P1: Alt paketlerde `file:..` bağımlılığı tarama ve dependency graph üzerinde recursive karmaşa yaratıyor.
- P1: Yazıcı profilleri env/JSON/DB seçeneklerine dağılmış. Kanıt: Electron bridge env üretimi `electron/main.cjs:211-226`, bridge normalize `server/routes/bridge.js:28-55`, renderer çözümleme `store-bridge/printers/renderers.js:253-287`.
- P2: Frontend test script'i yok. `client/package.json:6-9` yalnız `dev`, `build`, `preview` içeriyor.
- P2: Lint/typecheck script'i yok. Root scripts `package.json:8-37`, client scripts `client/package.json:6-9`, server scripts `server/package.json:6-10` içinde lint veya typecheck bulunmuyor.

- P2: Migration dosyası 673 satır ve hem schema hem veri düzeltme mantığı taşıyor. Uzun vadede migration sürümleme ve rollback disiplini zayıflar.
- P2: Runtime config kaynakları çok: `.env`, `pos-config.json`, Electron-generated env, bridge env, DB settings. Bu esneklik iyi ama tek doğruluk kaynağı belirsiz.

6. Hızlı kazanımlar

- P1: Server test kurulumunu düzelt: `npm install --prefix server`, ardından `npm test --prefix server`. CI yoksa minimum yerel gate olarak belgeye ekle.
- P1: Yazıcı encoding testlerini genişlet: `ÇĞİÖŞÜ çğİöşü`, TL sembolü, uzun ürün adı, sağ/sol hizalama ve farklı `escT` profilleri.
- P1: `client/src/components/orders/OrderScreen.jsx` için ilk parçalama: API orchestration hook, order total hesapları utility, UI alt komponentleri.
- P1: `store-bridge/printers/renderers.js` için ilk parçalama: `encoding`, `layout`, `escposCommands`, `templates`.
- P2: Vite code splitting: `xlsx`, reports/charts, settings/printer pages lazy load.
- P2: `docs/audit` altında her derin konu için ayrı rapor: `01-printing-flow.md`, `02-order-payment-integrity.md`, `03-frontend-state.md`, `04-packaging-electron.md`.

7. Uygulanmış düzeltmeler

- `docs/audit/00-overall-audit.md` oluşturuldu.
- Kod davranışı değiştirilmedi.
- Düşük riskli kod düzeltmesi uygulanmadı; çünkü bu turun açık kapsamı "ilk iş olarak sadece genel repo taraması" idi.

8. Çalıştırılan komutlar ve sonuçlar

- `Get-ChildItem -Force`: repo kök yapısı çıkarıldı. Ana klasörler: `client`, `server`, `electron`, `store-bridge`, `scripts`, `tools`, `resources`, `dist-electron`, `data`.
- `rg --files`: çalışmadı. Hata: WindowsApps içindeki `rg.exe` için erişim engellendi. Alternatif olarak PowerShell `Get-ChildItem` kullanıldı.
- `git status --short --branch`: branch `main...origin/main`. Mevcut kullanıcı/depo değişiklikleri görüldü: silinmiş `Restoran-POS-v3-Ilerleme-Raporu.html`, untracked `ANALIZ_RAPORU_2026.html`, untracked PDF. Bu dosyalara dokunulmadı.
- `Get-Content package.json`, `client/package.json`, `server/package.json`, `store-bridge/package.json`: script ve dependency yüzeyi incelendi.
- `Get-Content README.md -TotalCount 220`: ürün amacı, modlar ve kurulum bilgileri incelendi.
- `Get-Content server/index.js`, `client/src/App.jsx`, `client/src/services/api.js`, `server/config/database.js`: ana entrypoint ve request akışı incelendi.
- `Get-Content server/routes/orders.js`, `server/routes/payments.js`, `server/services/printJobs.js`, `server/routes/bridge.js`, `store-bridge/jobs/poller.js`, `store-bridge/printers/renderers.js`: sipariş, ödeme ve yazdırma akışı örnekledi.

- `npm test --prefix server`: başarısız. `vitest` komutu bulunamadı.
- `npm run build --prefix client`: başarılı. Vite build tamamlandı; büyük chunk uyarısı verdi.

9. Sonraki önerilen adımlar

1. `01-printing-flow.md`: Frontend'de yazdırma tetikleyen bütün UI noktaları, backend job üretimi, bridge claim/send akışı ve Türkçe karakter bozulma noktaları uçtan uca çıkarılmalı. Bu, ilk kullanıcı fotoğrafındaki gerçek problemi hedefler.
2. `02-order-payment-integrity.md`: Sipariş oluşturma, item ekleme/iptal/azaltma, ödeme kapatma, masa temizleme ve müşteri istatistiği transactional bütünlük açısından denetlenmeli.
3. `03-frontend-state.md`: `OrderScreen.jsx`, `PaymentScreen.jsx`, `App.jsx` state ve navigation davranışları çıkarılmalı; amatör/kırılgan UI koşulları listelenmeli.
4. `04-test-and-build-health.md`: Server test kurulumu onarılmalı, frontend test eksikliği ve CI gate önerisi raporlanmalı.
5. `05-electron-packaging.md`: `file:..` dependency döngüsü, dist-electron kaynak kopyaları, native sqllite rebuild ve bridge/caller ID process lifecycle denetlenmeli.